

Time in Distributed Computing

References

Kshemkalayani: chapter 3
Coulouris: chapter 14

Contents

1. Problem definition
2. Physical clocks synchronization
 - 2.1. Cristian algorithm
 - 2.2. Berkeley algorithm
 - 2.3. Network Time Protocol
3. Logical clocks
 - 3.1. Scalar time
 - 3.2. Vector time
 - 3.3. Efficient implementation of vector clocks - Singhal-Kshemkalyani

Problem definition

- Causality between events - fundamental issue for distributed / parallel systems
 - often in a distributed system it is necessary to be able to tell the order in which the events take place
- Causality (causal precedence relation) shows the relationship between two events (the cause and the effect)
 - the second event is a consequence of the first
- Causality is used in
 - distributed debugging
 - establishment of global breakpoints
 - implementations of causal ordering communication
 - determining the consistency of checkpoints in recovery

Problem definition

- Causality can be tracked using
 - Physical time
 - Synchronized computer clocks
 - Logical clock
 - scalar time
 - vector time
 - matrix time

Physical clock

- Two widely used (physical) time standards
 - atomic time: the frequency of atomic oscillations
 - almost zero clock drift
 - most accurate time
 - astronomical time: the rotation of the earth
 - rotation of earth is not uniform
- International Atomic Time (TAI)
 - high-precision atomic coordinate time standard based on the notional passage of proper time on Earth's geoid.
- Coordinate Universal Time (UTC)
 - international standard for timekeeping
 - based on atomic time, but with some insertion of leap seconds
 - broadcast from radio stations
 - computers with receivers synchronize their clocks with broadcast timing signals

Physical clock

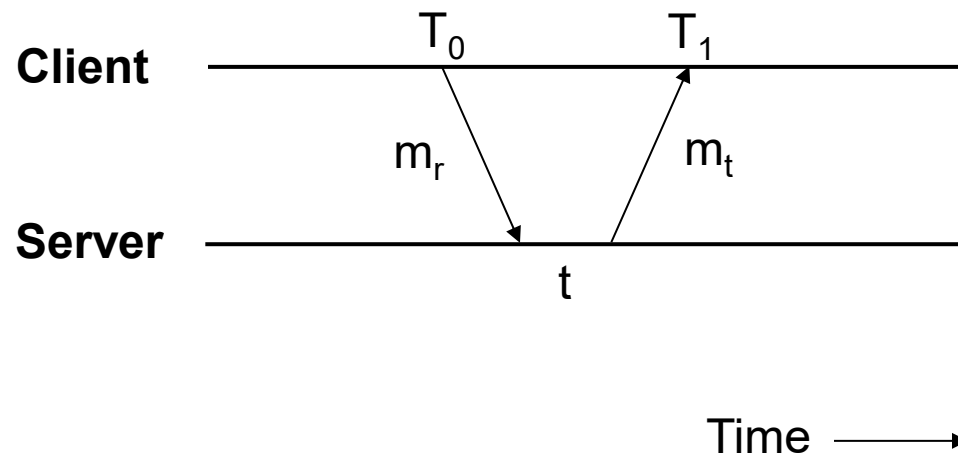
- Clock of computers may differ
 - difficult to set the same time for two clocks
 - suppose clocks are set accurately, but after a time due to the clock drift, they will differ → difficult to maintain synchronization between two clocks
- External synchronization
 - clock of a computer is synchronized with an external reference time source
- Internal synchronization
 - clocks of a collection of computers are synchronized with one another
- Local clock behind reference clock
 - local clock needs to “hurry up”
 - adjustment in one leap or in a sequence of smaller leaps
- Local clock ahead reference clock
 - local clock needs to “slow down”
 - adjustment by ignoring some clock ticks

Physical clock

- Why is clock synchronization needed?
 - analysis of the log information and debugging information collected from different devices in network management
 - all devices must use the same reference clock in a charging system
 - multiple systems process a complex event in cooperation
- Why clock synchronization is not easy?
 - most computers do not have receivers (due to the cost)
 - synchronization has to be done periodically due to the clock drift
 - if a computer clock is ahead of the reference, it cannot be simply set back
 - clock synchronization can be done only by message-passing

Physical clock - Cristian

- Time server maintains its clock using a UTC source
- A computer within the system synchronizes using the time server as reference

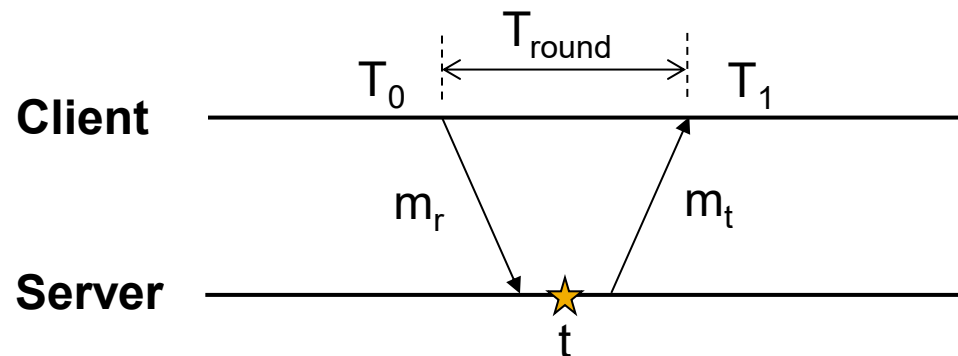


Physical clock - Cristian

- Client request the time by sending message m_r and gets time from the server in message m_t
- Client should set the clock to $t + T_{trans}$ where T_{trans} is time needed to transmit m_t from server to client
- T_{trans} varies and cannot be measured
- All we know about T_{trans} is
 - $T_{trans} = T_{min} + T_x$
 - T_{min} is time of message transmission in perfect conditions (e.g. no other messages send, no other requests to the server etc)

Physical clock - Cristian

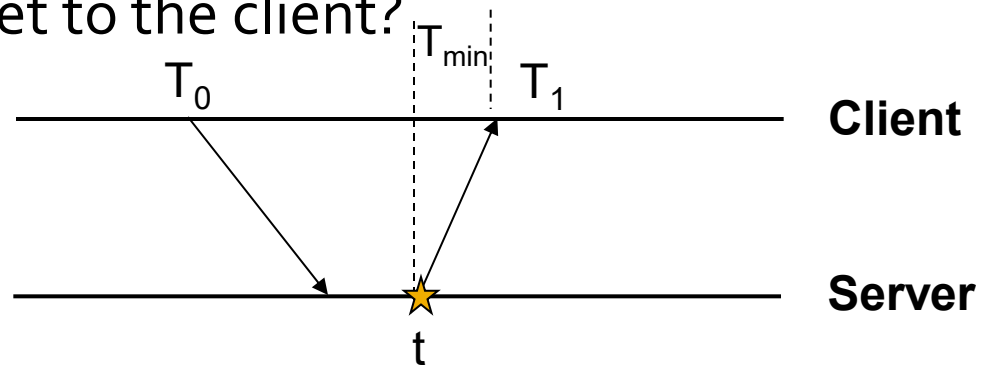
- Client records the total round-trip time T_{round}
 - $T_{\text{round}} = T_1 - T_0$
- When m_t is received by client, its clock is set to
 - $t_c = t + T_{\text{round}}/2$
 - based on the assumption that the elapsed time is equally divided before and after server put t in m_t



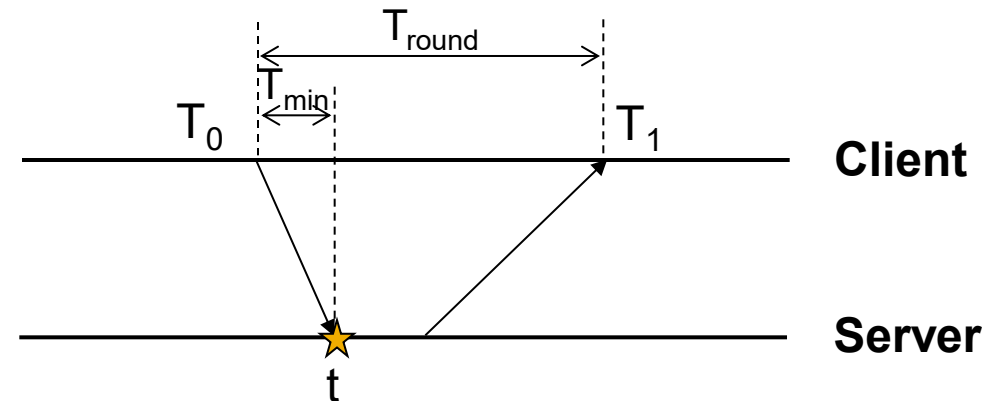
Physical clock - Cristian

- How accurate is the time set to the client?

$$t_c = t + T_{\min}$$



$$t_c = T_{\text{round}} - T_{\min} + t$$



Client clock $[t + T_{\min}, t + T_{\text{round}} - T_{\min}]$

Range width $T_{\text{round}} - 2T_{\min}$

Physical clock - Cristian

- Time at the client has an accuracy of

$$\pm((T_{\text{round}}/2) - T_{\text{min}})$$

- Client clock should be

$$t + (T_{\text{round}}/2) \pm ((T_{\text{round}}/2) - T_{\text{min}})$$

- How to increase the accuracy of client clock?
 - keep track of T_{round} and use the minimum value of T_{round}

Physical clock - Cristian

■ Disadvantages

- server is bottleneck / point of failure
 - use a collection of synchronized time servers
 - client sends the request to all servers and use the first reply
- malfunctioning / impostor time server
 - detection
 - authentication

Physical clock - Berkeley

- Time server doesn't maintain the reference clock
- Purpose of time server: maintain an average time as the global time
 - used to synchronize clocks that have different rates

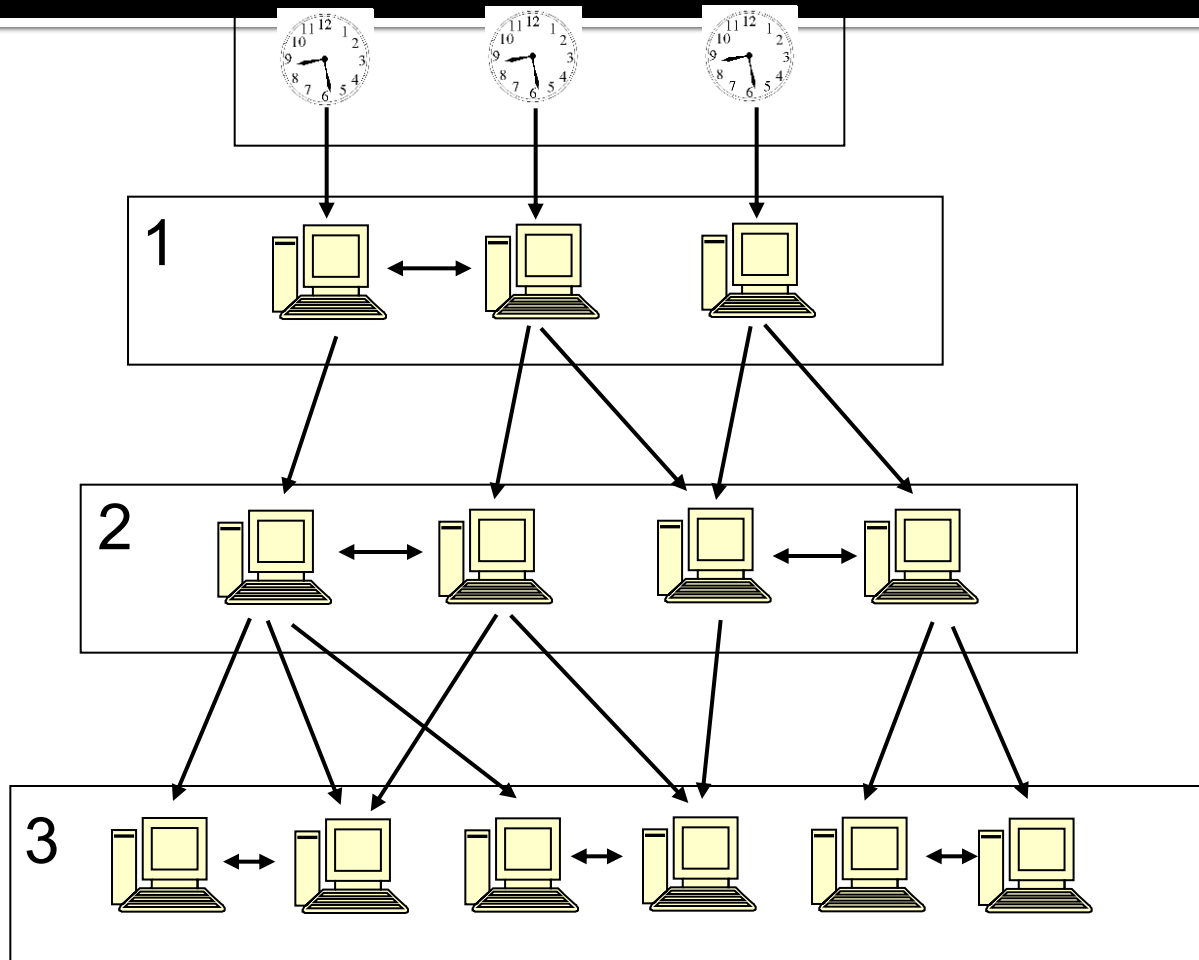
Physical clock - Berkeley

- A leader is elected every time time sync is needed
- Leader:
 - polls the other nodes to get their time
 - Clients reply with their time
 - estimates the nodes time as in Cristian's algorithm ($t + T_{\text{round}}/2$)
 - computes the average clock time
 - clocks whose values is outside an interval is not used for computing the average
 - sends back to the nodes the amount (positive or negative) that each must adjust its clock with

Physical clock

- Cristian: $t_c = t + T_{\text{round}}/2$
- Berkeley: $t_c = \text{average of computers clocks}$
- Is there a better way to compute the time?
- Better?
 - more accurate – for this you have to get a better estimates of delays
 - works despite the network failures
- NTP – Network Time Protocol

Physical clock - NTP



Physical clock - NTP

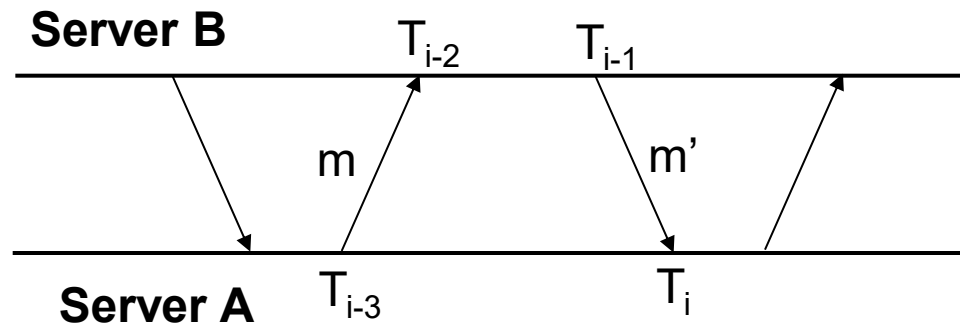
- Internet standard protocol
- Hierarchical organization in stratum
 - stratum 1 – primary servers (synchronized directly to reference clocks)
 - stratum 2 – synchronized with primary servers and between them
- If some servers fail the synchronization subnet can reconfigure
 - a primary that loses its source becomes secondary
 - a secondary that loses its primary can use another primary

Physical clock - NTP

- Methods for synchronization
 - Symmetric mode
 - Multicast mode
 - Procedure call mode
 - ...

Physical clock - NTP

- Symmetric mode
 - host sends NTP messages to make known that it is willing to be synchronized with and to be synchronized by its peer
 - typical scenario: a pair of servers exchange pairs of timing messages
 - each message sent contains timestamps of the previous message sent
 - highest accuracy



Physical clock - NTP

- Symmetric mode
 - pairs of offset and delay are stored
 - t is the transmission time for m
 - t' is the transmission time for m'
 - O is the offset between clock A and clock B
 - $A(t) = B(t) + O$ $A(t), B(t)$ clocks at A and B

$$T_{i-2} = T_{i-3} + t + O$$

$$T_i = T_{i-1} - O + t'$$

assume $t = t'$

$$O_i = (T_{i-2} - T_{i-3} + T_{i-1} - T_i)/2 \quad D_i = (T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$$

Physical clock - NTP

- Symmetric mode
 - the eight most recent pairs of (O_i, D_i) are stored
 - take minimum D_i - corresponding O_i will be used to estimate O_i
- a server interacts with several peers and identifies the most reliable ones
- accuracy 10s of msec / 1-50 msec over internet paths

Physical clock - NTP

- Multicast mode
 - used on high speed LAN
 - server sends time (at once) to all receivers in LAN
 - receivers reset their clocks
 - accuracy is not high
- Procedure call mode
 - clients send request to server
 - server replies with time
 - same behavior as Cristian
 - higher accuracy

Logical clocks

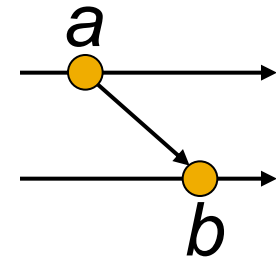
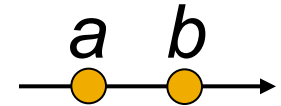
- Every process has a logical clock
 - logical clock assigns numbers to events, works as a counter, does not rely on physical time
 - every event has a timestamp
 - logical clock is modified following a set of rules
- Can causality relationship between events be inferred from their timestamps ? How?

Logical clocks

- Types of events in a distributed system:
 - Internal event: event that takes place at a process and does not affect other processes
 - Send event: a process transmits a message to another process
 - Receive event: a process gets a message from another process

Logical clocks

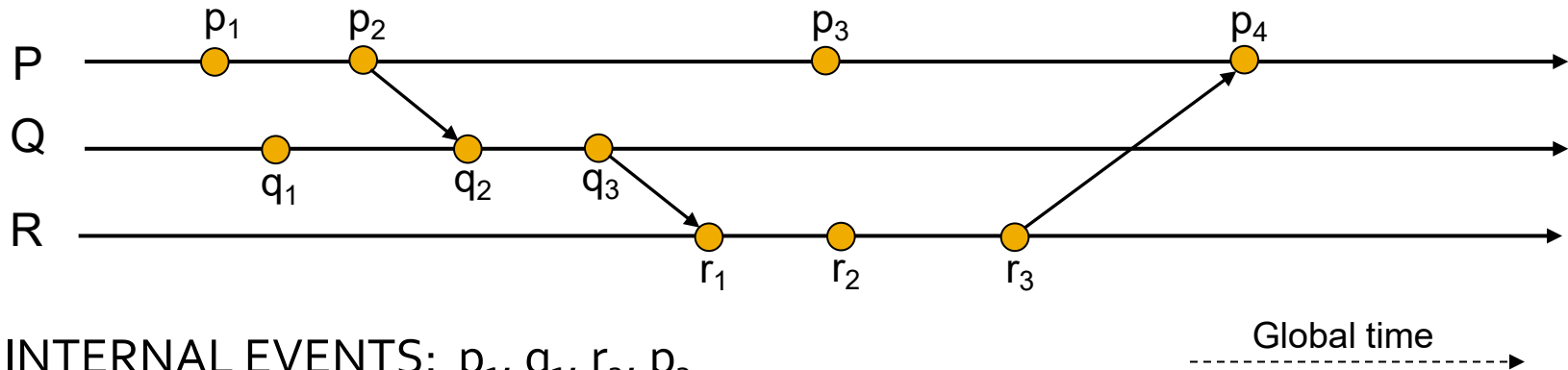
- Happened before relation is given by the following three rules:
 - if a and b are events in the same process and a comes before b , then a happened before b , $a \rightarrow b$
 - if a is sending of a message by one process and b is the receiving of the same message by another process then a happened before b , $a \rightarrow b$
 - if $a \rightarrow b$ and $b \rightarrow c$ then $a \rightarrow c$



Logical clocks

- Happened before between two events **may** (but not necessarily) imply a causality relationship between the two events
- Happened before relation is:
 - irreflexive: $a \not\rightarrow a$
 - asymmetric: $a \rightarrow b \Rightarrow b \not\rightarrow a$
 - transitive: $a \rightarrow b$ and $b \rightarrow c \Rightarrow a \rightarrow c$

Logical clocks



INTERNAL EVENTS: p_1, q_1, r_2, p_3

SEND EVENTS: p_2, q_3, r_3

RECEIVE EVENTS: q_2, r_1, p_4

CONCURRENT EVENTS: $p_1 \parallel q_1, \dots$

ORDERED EVENTS (happened before): $p_2 \rightarrow q_2, \dots$

Logical clocks

- System of logical clocks is consistent if
 - if event a happened before event b , then the timestamp of a is smaller than the timestamp of b
for two events a and b , $a \rightarrow b \Rightarrow C(a) < C(b)$
- System of logical clocks is strongly consistent if
 - if event a happened before event b , then the timestamp of a is smaller than the timestamp of b
 - if the timestamp of a is smaller than the timestamp of b then event a happened before event b
for two events a and b , $a \rightarrow b \Leftrightarrow C(a) < C(b)$

Logical clocks

- System of logical clocks is consistent if
 - if event a causally effects event b , then the timestamp of a is smaller than the timestamp of b
for two events a and b , $a \rightarrow b \Rightarrow C(a) < C(b)$
- System of logical clocks is strongly consistent if
 - if event a causally effects event b , then the timestamp of a is smaller than the timestamp of b
 - if the timestamp of a is smaller than the timestamp of b then event a causally effect event b
for two events a and b , $a \rightarrow b \Leftrightarrow C(a) < C(b)$

Logical clocks – scalar time (Lamport)

■ Implementation rules

- each process P_i increments C_i between any two successive events; if a and b are two successive events in P_i and $a \rightarrow b$, then

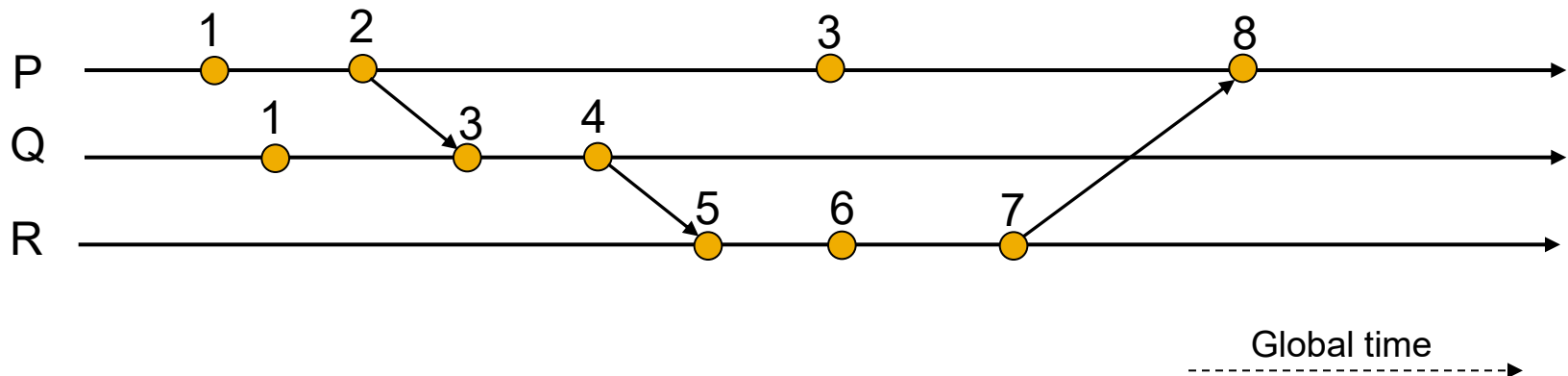
$$C_i(b) = C_i(a) + d, d > 0$$

- (a) a is event of sending of a message m by process P_i to P_j ; the message m contains a timestamp $T_m = C_i(a)$
(b) b is event of receiving of message m by P_j ; upon receiving m by process P_j :

$$C_j = \max(C_j, T_m) + d, d > 0$$

Logical clocks – scalar time

Implementing logical clocks



Logical clocks – scalar time

- Basic properties
 - total order
 - no strong consistency
 - event counting

Logical clocks – scalar time

- Total order
 - scalar time can be used to totally order events in a distributed system, if a tie-breaking mechanism is used
 - What “tie” is?
 - two or more processes within the distributed system can have identical timestamps $\Rightarrow$
$$e_1 \text{ and } e_2, C(e_1) = C(e_2) \quad e_1 \parallel e_2$$
 - tie-breaking mechanism is needed to order such events

Logical clocks – scalar time

- Total order - tie-breaking mechanism
 - tie is broken based on process identifier
 - events that occur at the same logical time are independent (not related) they can be ordered arbitrary
 - timestamp of an event is denoted by a tuple (t, i)
 - t is its time of occurrence
 - i is the identity of the process where it occurred
- total order relation $\prec$ on two events x and y with timestamps (h,i) and (k,j) , is defined as follows:
$$x \prec y \Leftrightarrow (h < k \text{ or } (h = k \text{ and } i < j))$$

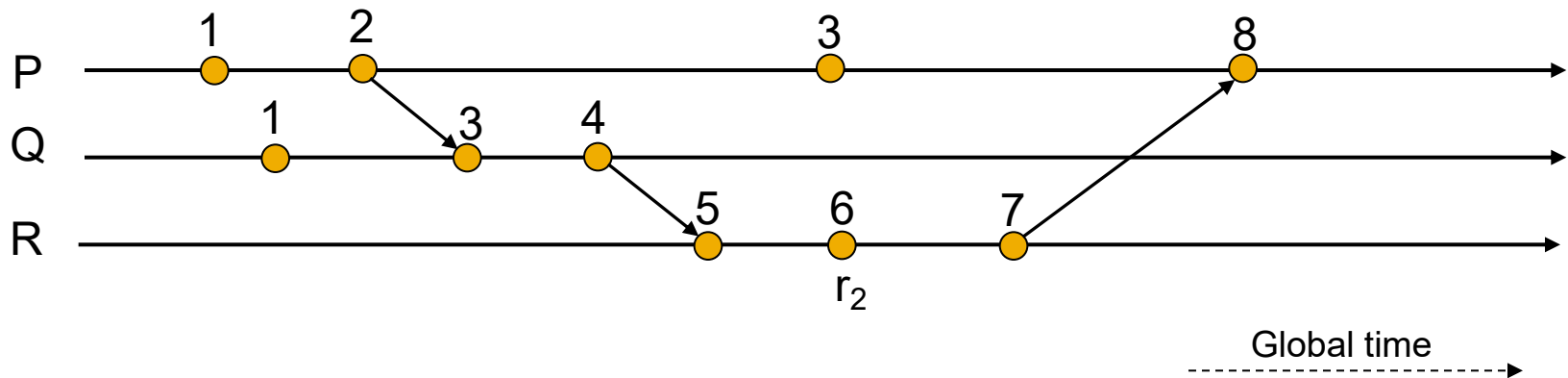
Logical clocks – scalar time

- No strong consistency
 - does not satisfy strong consistency property
for two events e_1 and e_2 , $C(e_1) < C(e_2) \implies e_1 \rightarrow e_2$
 - simply looking at the event clocks one cannot decide if the events are or are not causally related

Logical clocks – scalar time

■ Event counting

- if $d = 1$, e is the event with timestamp h then $h-1$ is the number of events that may causally produced the event e (at different processes)
- 5 events precede event r_2



Logical clocks – vector time

- Each process keeps a clock which is an integer vector of length n (n = no. of processes)
 - $VC_i[i]$ is the number of events that P_i has timestamped

Local logical clock

- $VC_i[j], j \neq i$ is P_i 's latest knowledge of P_j time
 - $VC_i[j] = x$, P_i knows that local time at P_j has progressed until x

Global logical clock

Logical clocks – vector time

■ Implementation rules

- each process P_i increments $VC_i[i]$ between any two successive events; if a and b are two successive events in P_i and $a \rightarrow b$, then

$$VC_i[i] = VC_i[i] + d, \quad d > 0$$

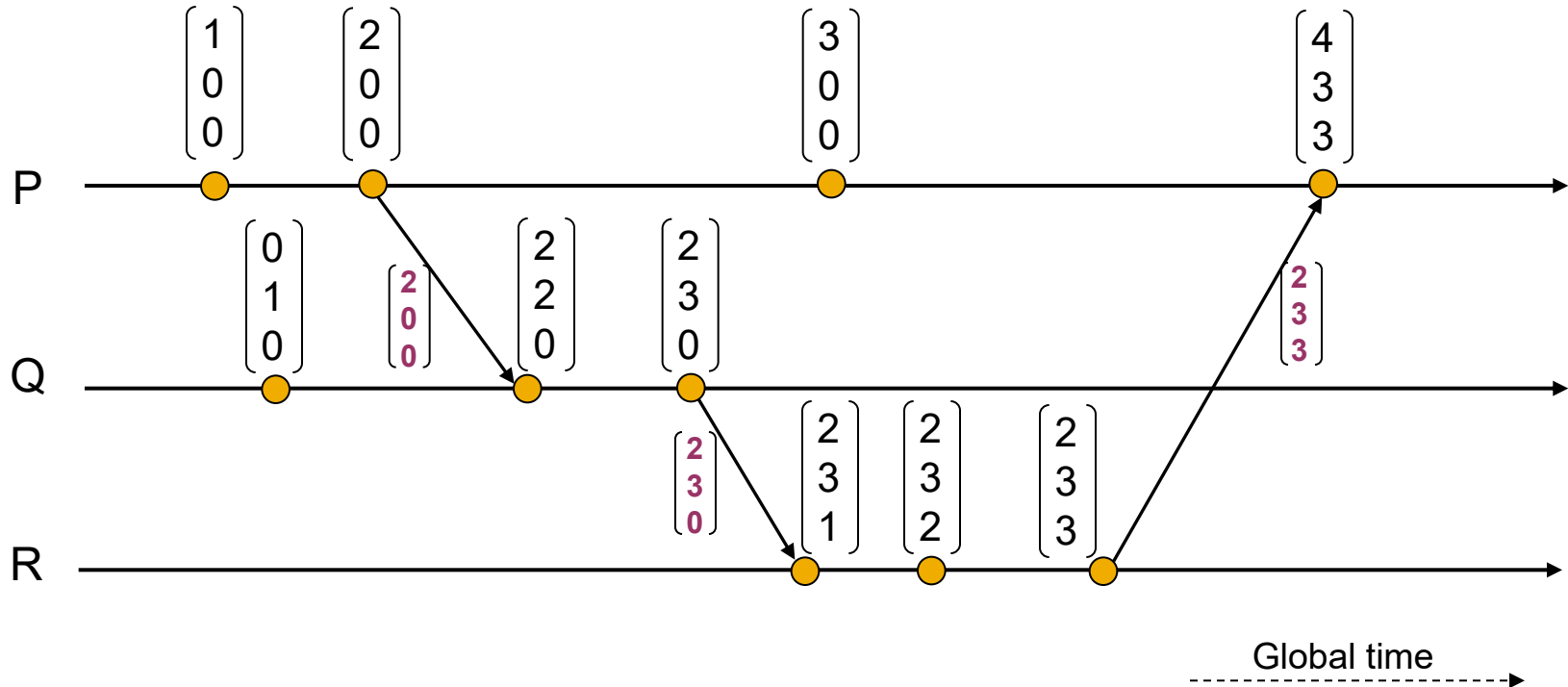
- (a) a is event of sending of a message m by process P_i to P_j ; the message m contains a timestamp $T_m = VC_i(a)$
(b) b is event of receiving of message m by P_j ; upon receiving m by process P_j :

$$VC_j[k] = \max(VC_j[k], T_m[k]), \text{ for } k \neq j$$

$$VC_j[j] = \max(VC_j[j], T_m[j]) + d, \quad d > 0$$

Logical clocks – vector time

- Implementing logical clocks
 - On the receiving of messages, a process learns about the more recent clock values of the rest of the processes in the system



Logical clocks – vector time

- Comparison between two vector clocks

$$VC_i = VC_j \Leftrightarrow \text{there is } x: VC_i[x] = VC_j[x]$$

$$VC_i \leq VC_j \Leftrightarrow \text{there is } x: VC_i[x] \leq VC_j[x]$$

$$VC_i < VC_j \Leftrightarrow VC_i \leq VC_j \text{ and there is } x: VC_i[x] < VC_j[x]$$

$$VC_i \parallel VC_j \Leftrightarrow \neg(VC_i < VC_j) \text{ and } \neg(VC_j < VC_i)$$

Logical clocks – vector time

- Basic properties
 - strong consistency: by examining the vector timestamp of two events, we can determine if the events are causally related
 - event counting
 - if $d = 1$, i^{th} component of vector clock at process P_i , $VC_i[i]$, denotes the number of events that have occurred at P_i until that instant
 - event e has timestamp VC , then $VC[k]$ denotes the number of events executed by process P_k that causally precede e

Logical clocks – efficient implementation

- Large number of processes leads to large vector clocks (size of vector clock = no of processors)
 - messages sent within distributed system are piggybacked with large vector clocks
- Straightforward implementation of vector clocks is inefficient
- How to efficiently implement vector clocks?
 - reduce vector clock size?
 - send only a part of the vector clock, not all? What part?

Logical clocks – efficient implementation

- For large system, between successive messages sent from P_i to P_j only a few entries of the vector clock VC_i change
 - Rarely: all entries of VC_i have been modified since last message, so the whole VC_i has to be sent
 - Often: only few entries of VC_i have been modified since last message, so these entries have to be sent
- When P_i sends a message to P_j , it sends in the timestamp T_m only the entries of VC_i that differ since the last message sent to P_j
 - differential technique
- When P_j receives the timestamp it updates its VC_j according to the second rule
- Disadvantage – each process has to keep the timestamp of the last message sent to each process
 - $O(n^2)$ storage overhead at each process

Singhal-Kshemkalyani

- How to reduce the storage overhead to $O(n)$?
- Each process P_i , apart from VC_i keeps two more vectors each of size n
 - LS (Last Sent)
 - LU (Last Update)
- How to use Last Sent and Last Update to figure out those entries of VC_i which differ since last message sent to P_j ?

Singhal-Kshemkalyani

$LS_i[j]$ = value of $VC_i[i]$ when process P_i last sent a message to P_j
 $LS_i[j]$ needs to be updated only when P_i sends a message to P_j

$LU_i[j]$ = value of $VC_i[i]$ when process P_i last updated the entry $VC_i[j]$
 $LU_i[j]$ needs to be updated only when the receipt of a message causes P_i to update $VC_i[j]$

If $LS_i[j] < LU_i[k]$ then $VC_i[k]$ has changed
since last message sent to P_j , therefore $VC_i[k]$
will be part of the timestamp

Singhal-Kshemkalyani

